

ARQUITECTURA MODERNIZACIÓN

Autor: Santiago Posada Zuluaga

Fecha: Julio 2026

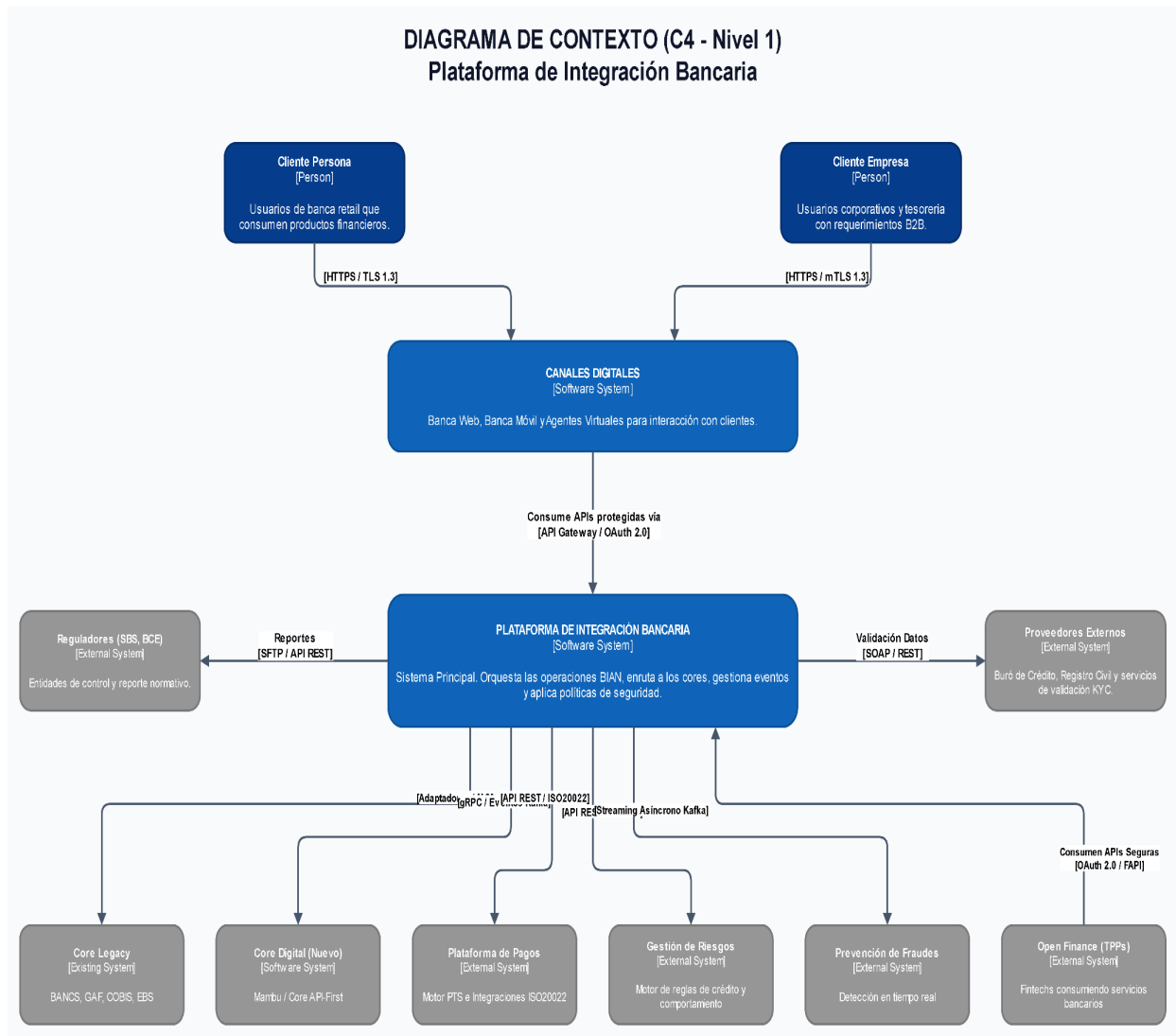
1. RESUMEN

Este documento presenta el diseño de una arquitectura de integración empresarial estratégica para impulsar la modernización de los sistemas de un banco tradicional. Con un enfoque centrado en la escalabilidad, la alta disponibilidad y la seguridad, el diseño se estructura sobre el estándar de capacidades de la industria **BIAN** (*Banking Industry Architecture Network*), una robusta **Arquitectura Orientada a Eventos (EDA)** y principios de **Clean Architecture**.

La solución emplea el patrón Strangler Fig para orquestar una coexistencia operativa transparente entre el Core Bancario Legacy y el nuevo Core Digital, erradicando el riesgo de una migración tipo Big-Bang. Asimismo, la plataforma unificada habilita canales digitales modernos, integraciones de Open Finance, sistemas de prevención de fraude en tiempo real y una telemetría avanzada que garantiza la observabilidad y trazabilidad integral del negocio (end-to-end).

2. DIAGRAMA C4 — NIVEL 1: CONTEXTO DEL SISTEMA

El diagrama de contexto establece los límites de la plataforma bancaria, sus usuarios e interacciones con el ecosistema externo (SaaS, reguladores, pasarelas de pago).

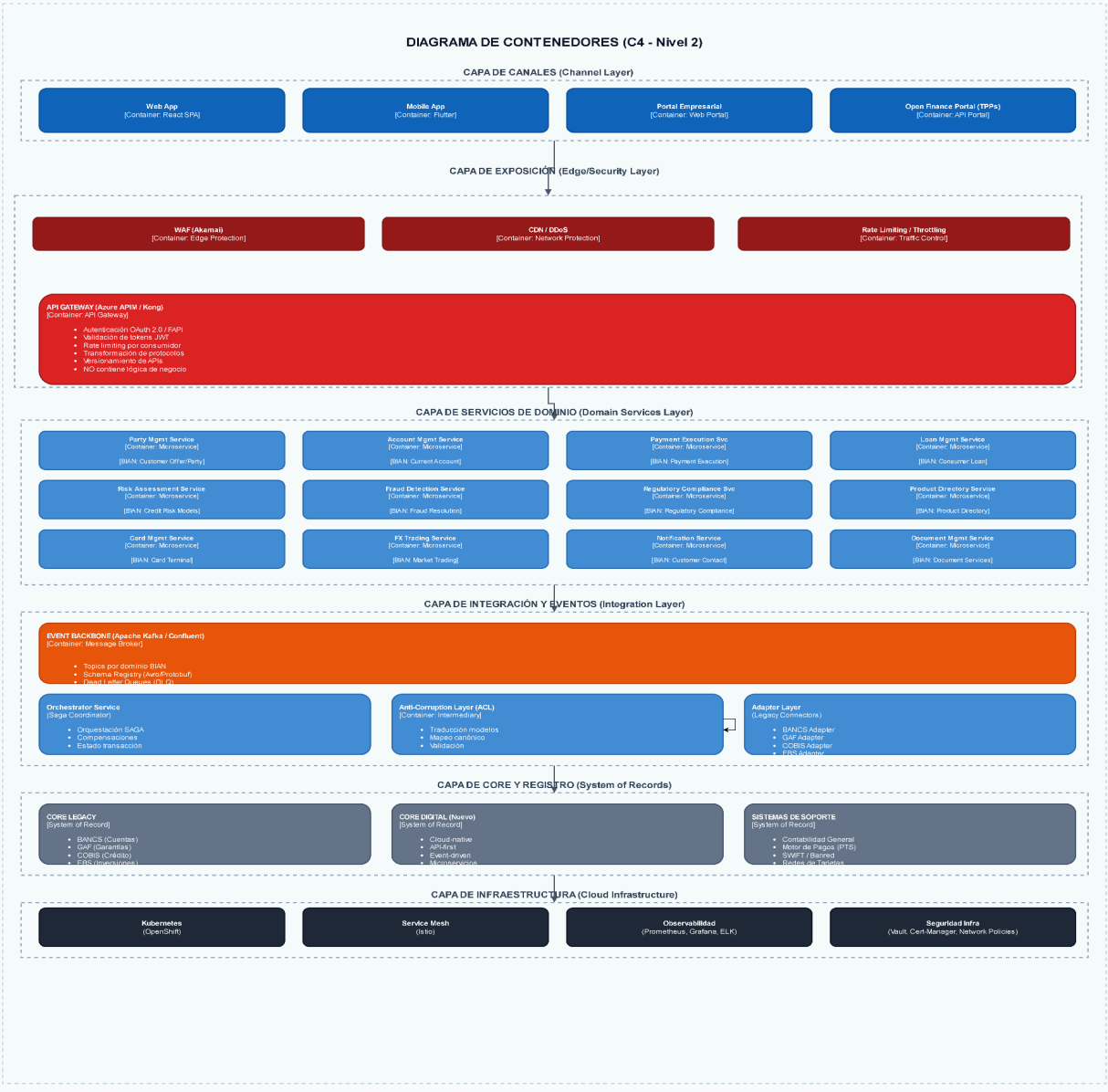


2.1 Actores y Sistemas Externos

Actor / Sistema	Tipo	Descripción	Protocolo
Cliente Persona Natural	Actor externo ▾	Usuarios de banca retail	HTTPS, OAuth 2.0
Cliente Empresa	Actor externo ▾	Usuarios corporativos	HTTPS, mTLS, OAuth 2.0
Open Finance (TPPs)	Sistema exte... ▾	Fintechs y terceros autorizados	API REST, OAuth 2.0 + FAPI
Reguladores (SBS, BCE, SEPS)	Sistema exte... ▾	Organismos de control financiero	SFTP, API REST, ISO 20022
Proveedores de datos	Sistema exte... ▾	Buró de crédito, Registro Civil	API REST, SOAP
Core Bancario Legacy	Sistema inter... ▾	Cores heredados (BANCS, COBIS)	Adaptadores, TCP, SOAP
Core Bancario Digital	Sistema inter... ▾	Nuevo core cloud-native	API REST, gRPC, Eventos
Plataforma de Pagos (PTS)	Sistema inter... ▾	Motor de pagos	API REST, ISO 20022, Eventos

3. DIAGRAMA C4 — NIVEL 2: CONTENEDORES (Arquitectura BIAN)

La Plataforma de Integración Bancaria ha sido descompuesta en un modelo de capas estrictas, garantizando la separación de responsabilidades.

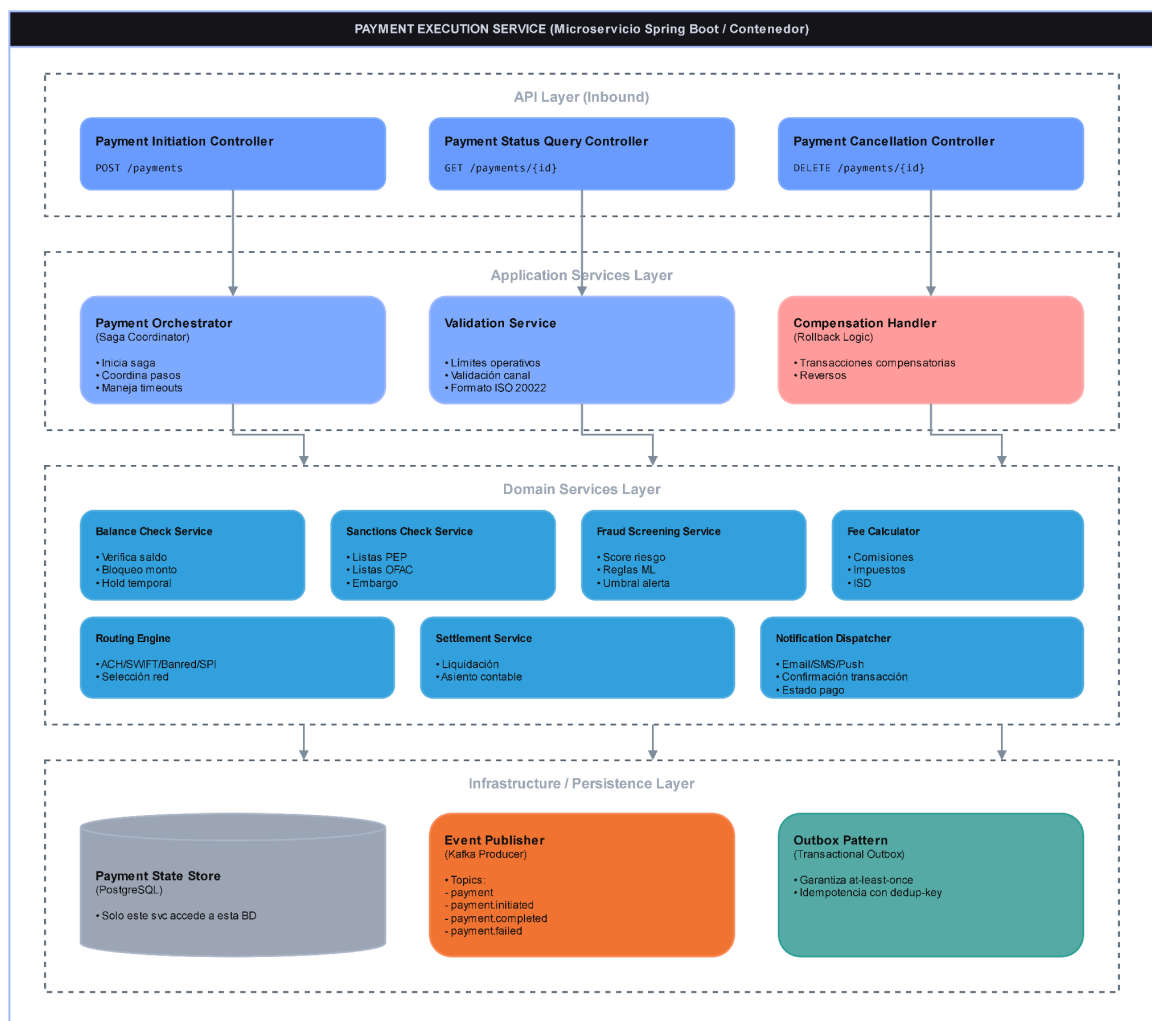


- **Capa de Canales (Channel Layer):** Aplicaciones nativas (Banca Móvil), portales web, y portales empresariales.
- **Capa de Exposición (Edge & Security Layer):** WAF (Akamai), CDN, DDoS protection. Incluye el **API Gateway** que maneja Rate Limiting y Validación JWT. *No contiene lógica de negocio.*
- **Capa de Servicios de Dominio:** Microservicios agrupados por dominios **BIAN** (Party Mgmt, Account Mgmt, Payment Execution, Risk Assessment).
- **Capa de Integración y Eventos: Event Backbone (Kafka)** con Schema Registry, colas de mensajes no entregados (DLQ) y el servicio de Orquestación (SAGA). Aquí vive la **Anti-Corruption Layer (ACL)**.
- **Capa de Core y Registro (SoR):** Core Legacy, Nuevo Core Digital y Sistemas de Soporte (Stratio Data Fabric, PTS).
- **Capa de Infraestructura (Cloud):** Kubernetes (OpenShift), **Istio Service Mesh**, Seguridad (Vault), Observabilidad (Prometheus, ELK, Grafana).

4. DIAGRAMA C4 — NIVEL 3: COMPONENTES

4.1 Payment Execution Service (Dominio BIAN: Payment Order)

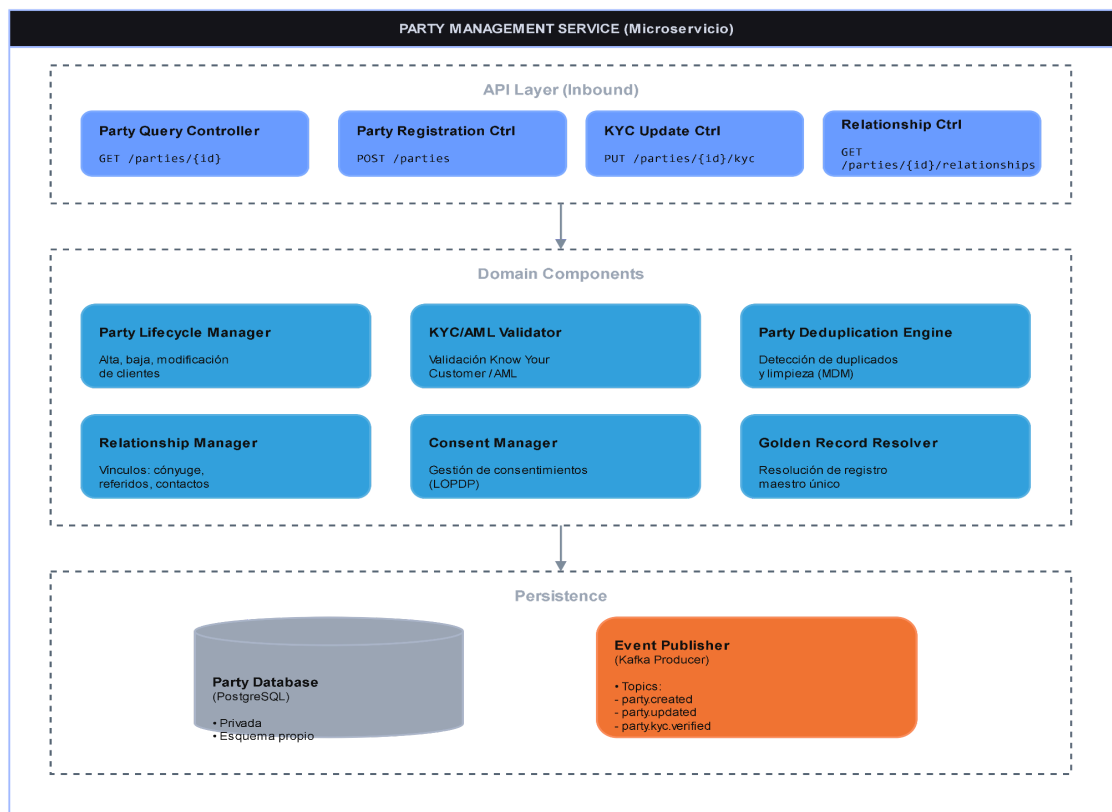
- **Inbound:** POST /payments.
- **Application Services:** Payment Orchestrator (Saga Coordinator), Compensation Handler (maneja los rollbacks asíncronos en caso de fallas temporales de red o fondos).
- **Domain Services:** Balance Check, Sanctions Check, Routing Engine y Settlement Service.
- **Infrastructure / Persistence:** Base de datos dedicada en PostgreSQL. Implementa el patrón **Transactional Outbox** emitiendo eventos (payment.initiated, payment.completed) hacia Kafka garantizando semántica *at-least-once*.



4.2 Party Management Service (Dominio BIAN: Customer Profile)

Actúa como el núcleo central para la gestión de clientes bajo el estándar BIAN. Sus características más críticas son:

- **Gestión del "Golden Record":** A través de sus componentes de dominio (como el *Deduplication Engine* y el *Golden Record Resolver*), este servicio se asegura de que exista una única versión de la verdad (Dato Maestro) para cada cliente, evitando registros duplicados.
- **Cumplimiento Normativo Integrado:** Incluye validadores específicos para **KYC/AML** (Conozca a su Cliente / Prevención de Lavado de Activos) y un **Consent Manager**, que es la pieza fundamental para garantizar el cumplimiento de la **LOPD** (Ecuador) registrando los consentimientos legales.
- **Aislamiento y Eventos (EDA):** Respeta el principio de *Database-per-Service* utilizando una base de datos PostgreSQL privada con esquema propio. Además, propaga asíncronamente cualquier cambio en el ciclo de vida del cliente (creación, actualización, verificación) hacia el resto de la plataforma mediante un publicador de eventos (*Event Publisher*).

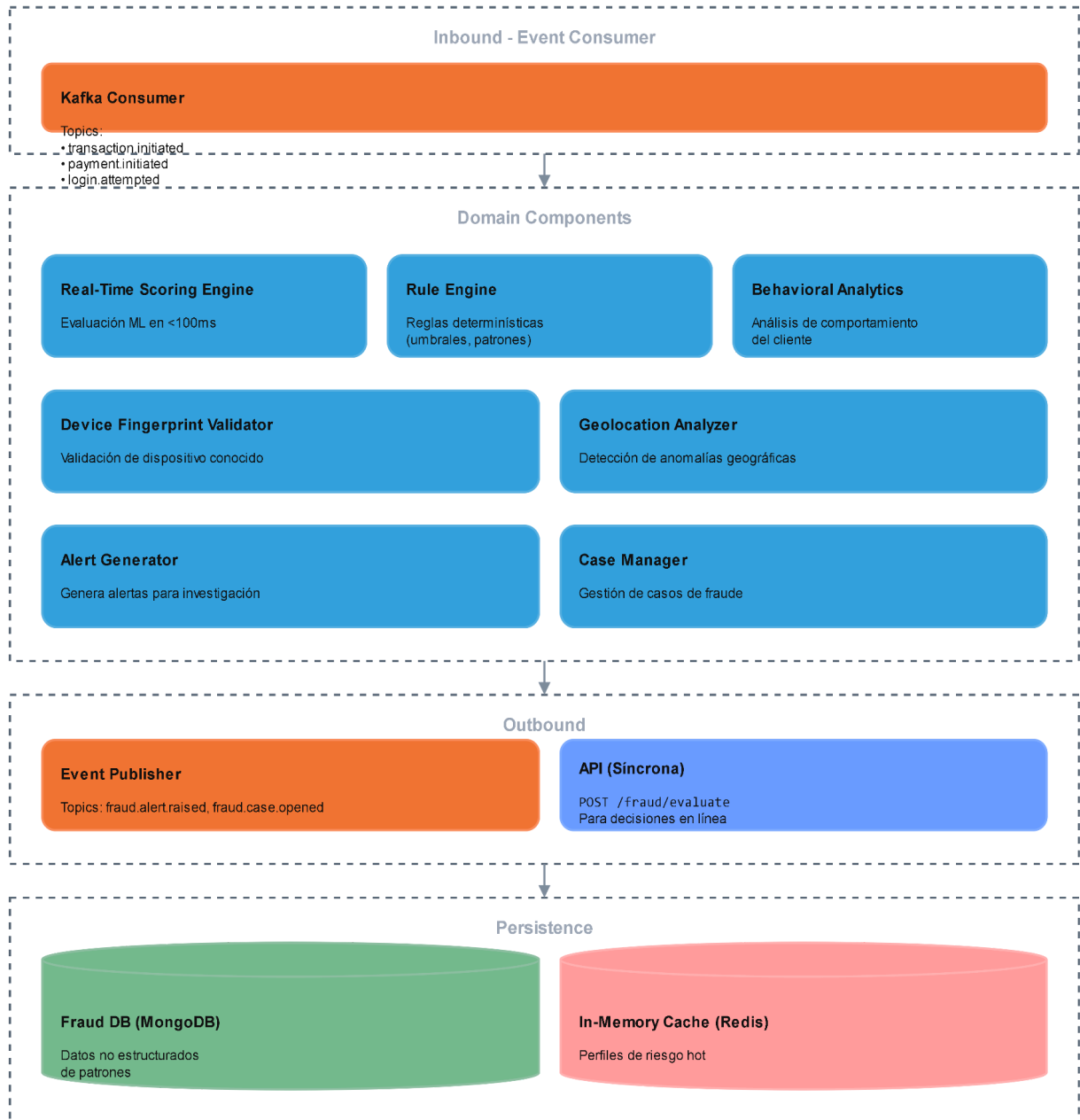


4.3 Fraud Detection Service (Dominio BIAN: Fraud Resolution)

Componente de misión crítica diseñado para operar en tiempo real bajo el estándar BIAN. Sus características principales son:

- **Arquitectura Orientada a Eventos (EDA) de Alta Velocidad:** Opera primariamente como un consumidor de Kafka, ingiriendo eventos como intentos de login o inicios de transacciones.
- **Evaluación de Sub-milisegundo:** El núcleo del dominio combina un *Real-Time Scoring Engine* (modelos de Machine Learning) y un *Rule Engine* determinístico, logrando evaluar el riesgo y detectar anomalías de geolocalización o dispositivos en menos de 100 milisegundos.
- **Persistencia Políglota:** Adopta el almacenamiento adecuado para cada necesidad: utiliza **MongoDB** para guardar patrones y datos no estructurados de fraude, y **Redis** como caché en memoria para acceder instantáneamente a perfiles de riesgo "calientes" (*hot data*).
- **Salida Dual (Síncrona/Asíncrona):** Emite alertas de forma asíncrona hacia otros tópicos de Kafka ([fraud.alert.raised](#)), pero también expone un endpoint síncrono ([POST /fraud/evaluate](#)) para decisiones que bloquean transacciones en línea.

FRAUD DETECTION SERVICE (Microservicio)



5. PATRONES DE INTEGRACIÓN Y TECNOLOGÍAS

5.1 Patrones Aplicados con Justificación

La adopción de patrones de diseño de integración en esta arquitectura responde a la necesidad de aislar la complejidad, garantizar la consistencia transaccional y escalar los componentes de forma independiente.

Patrón	Dónde se aplica	Por qué se aplica	Tecnología
Anti-Corruption Layer (ACL)	Entre los servicios de dominio BIAN y los Cores Legacy (BANCS, GAF, COBIS).	Protege el modelo de dominio moderno de la deuda técnica y los formatos propietarios del sistema heredado, traduciendo hacia el modelo canónico BIAN.	Microservicios adaptadores en Red Hat OpenShift
Saga (Orquestación)	En transacciones distribuidas complejas que cruzan múltiples dominios (ej. apertura de depósitos, transferencias).	Garantiza la consistencia eventual sin recurrir a costosas transacciones distribuidas bloqueantes (2PC), permitiendo disparar compensaciones explícitas ante fallos de red.	Orquestador dedicado con state store en PostgreSQL

Patrón	Dónde se aplica	Por qué se aplica	Tecnología
Event Sourcing	En los servicios de transacciones y estados (Payment Execution, Account Management).	Permite reconstruir el estado actual a partir de la secuencia inmutable de eventos, facilitando la auditoría completa y la recuperación (replay).	Apache Kafka como Event Store
CQRS	En consultas de saldos, estados de cuenta y reportes analíticos regulatorios.	Separa los modelos de lectura (optimizados para alta concurrencia y consultas rápidas) de los modelos de escritura (optimizados para consistencia transaccional).	Read replicas y vistas materializadas
Transactional Outbox	En todos los microservicios que publican eventos hacia el bus de mensajería.	Garantiza la atomicidad entre la persistencia de los datos en la base de datos local y la publicación efectiva del evento (at-least-once delivery).	Debezium CDC + Kafka Connect
Database per Service	En cada microservicio de dominio de la plataforma.	Asegura el aislamiento de datos y evita el acoplamiento a nivel de bases de datos compartidas, permitiendo la evolución	PostgreSQL (transaccional), MongoDB (documentos), Redis (caché)

Patrón	Dónde se aplica	Por qué se aplica	Tecnología
		independiente de los esquemas.	
API Gateway	Como punto de entrada único perimetral para todos los canales de consumo.	Centraliza las tareas transversales de infraestructura de red y seguridad perimetral (auth, rate limiting, logging) sin contaminar el negocio.	Azure APIM / Kong Enterprise
Circuit Breaker	En la comunicación síncrona entre microservicios y conectores hacia sistemas externos.	Previene fallas en cascada y bloqueos de hilos cuando un servicio descendente sufre degradación o caídas de rendimiento (fail-fast).	Istio Service Mesh + Resilience4j
Strangler Fig	En la estrategia de convivencia del ecosistema Multicore.	Facilita la migración gradual de funcionalidades del Core Legacy al Core Digital sin arriesgar operaciones mediante un big-bang.	Routing rules en API Gateway + Feature Flags
Event-Driven (Pub/Sub)	En la comunicación asíncrona entre dominios de negocio desacoplados.	Proporciona desacoplamiento temporal y espacial, permitiendo que productores y consumidores escalen y evolucionen por separado.	Apache Kafka con Schema Registry

Patrón	Dónde se aplica	Por qué se aplica	Tecnología
Canonical Data Model	En toda la plataforma central de intercambio de mensajes.	Establece un vocabulario de datos estandarizado basado en BIAN que elimina la necesidad de traducciones punto a punto.	Esquemas Avro o Protobuf versionados

5.2 Principios de Diseño Reflejados en la Arquitectura

Los principios SOLID y las directrices de Clean Architecture se manifiestan de manera explícita en los límites y contratos de los servicios propuestos:

Principio	Cómo se refleja en la arquitectura
Responsabilidad Única (SRP)	Cada microservicio se restringe estrictamente a una capacidad funcional del dominio BIAN (Ej. Payment Execution procesa transacciones monetarias, pero no valida riesgos ni gestiona identidades de usuario).
Segregación de Interfaces (ISP)	Los adaptadores del API Gateway y los contratos BFF exponen esquemas adaptados al perfil del consumidor (ej. interfaces simplificadas para banca retail móvil en comparación con pasarelas robustas de FAPI para Open Finance).
Inversión de Dependencias (DIP)	Los microservicios de la capa de aplicación y negocio dependen exclusivamente de abstracciones y contratos estipulados, aislando los detalles de infraestructura y base de datos.
Abierto/Cerrado (OCP)	La incorporación de nuevas capacidades se realiza mediante la creación de nuevos microservicios desacoplados o la suscripción a tópicos de eventos existentes, sin alterar el código base en producción.

Principio	Cómo se refleja en la arquitectura
Bajo Acoplamiento	Las dependencias directas entre sistemas se reducen al mínimo mediante comunicación asíncrona basada en eventos sobre Kafka y aislamiento físico de bases de datos por servicio.
Alta Cohesión	Toda la lógica operativa vinculada a una capacidad de negocio se agrupa dentro de su respectivo microservicio de dominio BIAN.
Contratos Explícitos	Las interfaces REST se documentan estrictamente bajo OpenAPI 3.0, mientras que los eventos de mensajería validan sus cargas contra esquemas semánticos en el Schema Registry.
Testabilidad	El aislamiento de la lógica de negocio frente a frameworks externos permite realizar pruebas unitarias, de integración efímeras y pruebas de contrato con Pact.

6. SEGURIDAD, CUMPLIMIENTO NORMATIVO Y LOPDP

6.1 Ley Orgánica de Protección de Datos Personales (LOPDP — Ecuador)

La arquitectura implementa el principio de *Security & Privacy by Design*, reflejando de forma práctica las exigencias normativas del tratamiento de datos personales:

Requisito LOPDP	Implementación en la Arquitectura	Componente Responsable
Consentimiento informado (Art. 7)	Módulo dedicado que captura, versiona, audita y permite revocar los consentimientos de tratamiento de datos.	Party Management → Consent Manager
Finalidad (Art. 8)	Los contratos de API especifican el uso autorizado; el API Gateway rechaza	API Gateway + OAuth Scopes

Requisito LOPDP	Implementación en la Arquitectura	Componente Responsable
	peticiones con scopes no válidos.	
Minimización de datos (Art. 10)	Las respuestas aplican filtrado de atributos y transformación de payload según los privilegios del consumidor.	API Gateway + ISP en servicios
Exactitud (Art. 11)	Componente encargado de unificar los datos maestros de contraparte para impedir discrepancias o duplicados.	Party Management → Golden Record Resolver
Limitación de conservación (Art. 12)	Políticas automáticas de retención de registros de auditoría y ciclos de vida configurables mediante TTL.	Data Lifecycle Manager
Seguridad del tratamiento (Art. 37)	Cifrado obligatorio en tránsito (mTLS) y en reposo (AES-256), complementado con tokenización de datos.	Service Mesh (Istio) + Vault + DB Encryption
Notificación de brechas (Art. 40)	Monitoreo continuo de accesos anómalos o fugas de información que dispara alertas tempranas.	Observability Stack + Alert Manager
Derechos ARCO (Acceso, Rectificación, Cancelación, Oposición)	Endpoints específicos y dedicados que permiten al titular gestionar su información personal.	Party Management Service
Transferencia internacional (Art. 42)	Restricciones estrictas de conectividad perimetral y zonificación geográfica de almacenamiento en la nube.	Network Policies + Cloud Region Constraints

Requisito LOPDP	Implementación en la Arquitectura	Componente Responsable
Evaluación de impacto (Art. 34)	Incorporación de revisiones de privacidad obligatorias automatizadas como parte de las puertas de despliegue.	Governance Framework + CI/CD

6.2 Tabla de Controles de Seguridad Perimetral e Interna

Capa	Control	Implementación	Ubicación en Arquitectura
Perímetro	WAF	Akamai WAF — protección contra ataques web comunes y gestión avanzada de bots.	Edge Layer (antes del API Gateway)
Perímetro	DDoS Protection	Akamai + Azure DDoS Protection para mitigación volumétrica.	Edge Layer
Transporte	TLS 1.3	Cifrado robusto en tránsito para todas las comunicaciones de cara al cliente.	Entre clientes y API Gateway
Transporte	mTLS	Autenticación mutua obligatoria para aislar y blindar el tráfico entre microservicios.	Service Mesh (Istio) — Zero Trust
Autenticación	OAuth 2.0 + OIDC	Emisión de tokens firmados portadores de claims de identidad y permisos.	Identity Provider → API Gateway
Autenticación	FAPI (Financial-grade API)	Perfil endurecido de seguridad para	API Gateway

Capa	Control	Implementación	Ubicación en Arquitectura
		operaciones financieras abiertas (Open Finance).	
Autenticación	MFA	Autenticación de múltiples factores para transacciones monetarias o sensibles.	Identity Service + Canales
Autorización	RBAC + ABAC	Control de acceso basado en roles y atributos contextuales del usuario o canal.	Policy Decision Point (OPA)
Datos en reposo	AES-256	Cifrado simétrico de bases de datos operativas y volúmenes de respaldo.	Capa de persistencia por microservicio
Datos sensibles	Tokenización	Reemplazo de números de tarjetas (PAN) e identificadores por tokens seguros.	Tokenization Service
Datos sensibles	Data Masking	Enmascaramiento dinámico de información sensible en entornos de pruebas.	CI/CD Pipeline
Secretos	Vault	Almacenamiento seguro, inyección y rotación programada de credenciales.	HashiCorp Vault
API	Rate Limiting	Regulación de tráfico y cuotas máximas por consumidor o partner.	API Gateway (Azure APIM)

Capa	Control	Implementación	Ubicación en Arquitectura
API	Input Validation	Comprobación de integridad y esquemas de entrada en los límites de servicio.	Capa API de cada microservicio
Auditoría	Audit Trail	Registro inmutable de transacciones financieras y de datos personales.	Audit Service → Event Store
Red	Network Policies	Aislamiento físico de tráfico lógico no autorizado entre contenedores.	Kubernetes Network Policies + Istio
Código	SAST/DAST	Análisis estático y dinámico de vulnerabilidades en el código fuente.	CI/CD Pipeline

7. ALTA DISPONIBILIDAD (HA) Y RECUPERACIÓN ANTE DESASTRES (DR)

7.1 Estrategia HA/DR Adaptada al Diseño

Resiliencia geográfica de la plataforma basada en un modelo activo-activo multi-zona para la región primaria y replicación multi-región para la contingencia. Sus elementos más destacados son:

- **Topología Multi-Región Activo-Standby:** La operación principal recae en la **Región Primaria (Quito)** utilizando múltiples Zonas de Disponibilidad (AZ-1 y AZ-2) con clústeres de cómputo en modo activo simultáneo. La **Región Secundaria (Guayaquil)** opera en modo *Standby* pasivo/activo listo para tomar el tráfico transaccional en caso de un desastre regional.
- **Replicación Asíncrona Sincronizada con MirrorMaker2:** El bus de eventos dorsal (**Apache Kafka**) se despliega con un factor de replicación de 3 (**RF=3**) y réplicas sincronizadas dentro de la región primaria, utilizando *Confluent MirrorMaker2* para replicar los streams de datos de manera fluida y con mínima latencia hacia el clúster de respaldo en la región secundaria.

- **Persistencia Tolerante a Fallos:** Las bases de datos transaccionales (**PostgreSQL**) implementa un esquema de *Sync Standby* local entre zonas para garantizar cero pérdida de datos locales (**RPO = 0**), respaldado por una réplica asíncrona cruzada hacia la región secundaria para la recuperación ante desastres globales.



7.2 Parámetros de Disponibilidad

Componente	RPO	RTO	Estrategia
API Gateway	0	<30s	Active-Active multi-AZ, failover automático
Microservicios (stateless)	N/A	<60s	Múltiples réplicas en diferentes AZ, auto-scaling
Kafka (Event Backbone)	0	<2min	Replication Factor=3, min.insync.replicas=2, MirrorMaker2 cross-region
PostgreSQL (transaccional)	<5s	<5min	Synchronous standby intra-región + async cross-región
Redis (cache)	N/A	<30s	Cluster mode con réplicas, rehidratación desde event store
Core Legacy	Depende del core	Depende del core	Mecanismos propios del core + circuit breaker en adaptadores
Core Digital	<1s	<2min	Cloud-native, multi-AZ, auto-healing

7.3 Mecanismos Específicos

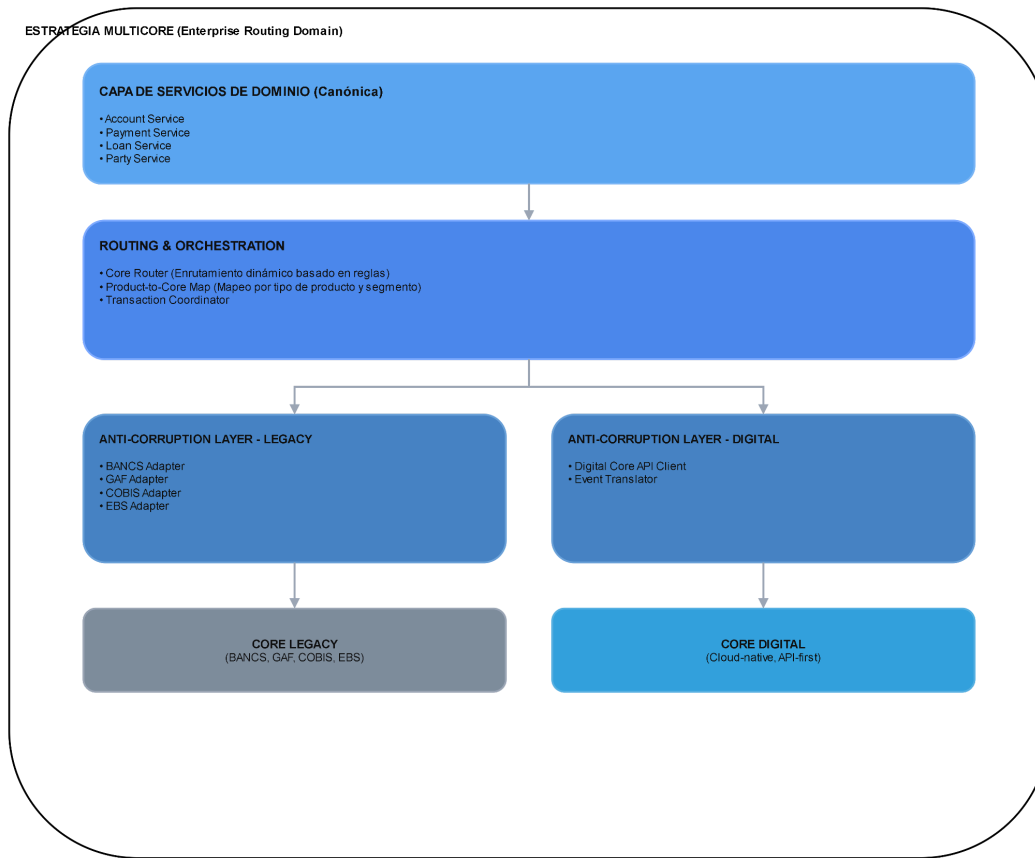
1. **Circuit Breaker en Adaptadores Legacy:** Si el core BANCS no responde en 3s, el circuit breaker se abre, las transacciones se encolan en Kafka y se reprocesan cuando el core se recupera (retry con backoff exponencial).
2. **Event Replay:** Gracias a Event Sourcing, ante una falla catastrófica de un servicio, se puede reconstruir su estado completo reproduciendo eventos desde el Event Store.
3. **Bulkhead Pattern:** Cada dominio tiene pools de conexiones y threads independientes. Una degradación en Fraud Detection no afecta a Payment Execution.
4. **Health Checks y Auto-healing:** Kubernetes realiza liveness/readiness probes. Pods no saludables se reemplazan automáticamente.
5. **Failover DNS:** Global Traffic Manager (GTM) redirige tráfico a la región secundaria si la primaria no responde en <30s.

8. ESTRATEGIA DE INTEGRACIÓN MULTICORE

Basada en la separación del modelo de dominio y la traducción perimetral mediante capas de anticorrupción independientes para los sistemas heredados y digitales. Sus elementos más destacados son:

- **Enrutamiento y Orquestación Centralizada:** El flujo desciende desde la *Capa de Servicios de Dominio (Canónica)* hacia el bloque de *Routing & Orchestration*, el cual contiene el **Core Router**, el **Product-to-Core Map** y el **Transaction Coordinator** para derivar inteligentemente las peticiones de negocio.
- **Doble Capa de Anticorrupción (ACL):** Se divide simétricamente el tráfico saliente:
 - *ACL Legacy:* Utiliza adaptadores especializados como el *BANCS Adapter*, *GAF Adapter*, *COBIS Adapter* y *EBS Adapter* para comunicarse de manera segura con el *Core Legacy*.
 - *ACL Digital:* Emplea un *Digital Core API Client* y un *Event Translator* para interactuar nativamente con el nuevo *Core Digital* cloud-native.
- **Convivencia Armónica (SoR):** Permite procesar transacciones concurrentes dirigidas al ecosistema tradicional o al moderno sin afectar los contratos canónicos expuestos en la capa superior de servicios.

8.1 Modelo de Convivencia



8.2 Principios de la Estrategia Multicore

- **Core Router (Enrutador de Core)**: Componente que determina a qué core dirigir una operación basándose en:
 - Tipo de producto (ej: cuentas de ahorro nuevas → Core Digital; cuentas legacy existentes → BANCS)
 - Segmento de cliente (ej: clientes digitales nuevos → Core Digital)
 - Tipo de operación (ej: consultas → CQRS read model; transacciones → core correspondiente)
- **Product-to-Core Mapping**: Tabla de configuración que mapea cada producto bancario al core que lo procesa. Permite migrar productos gradualmente cambiando solo la configuración.
- **Modelo Canónico Único**: Los servicios de dominio trabajan con un modelo de datos canónico basado en BIAN. Los adaptadores (ACL) traducen desde/hacia los modelos propietarios de cada core.
- **Consolidación de Saldos**: Un servicio de "Account Aggregation" consulta ambos cores y presenta una vista unificada al cliente.

- **Sincronización de Datos Maestros:** El Party Management Service es el golden source de datos de cliente. Propaga cambios a ambos cores vía eventos.

8.3 Reflejo en el Diseño

- **En el Diagrama de Contenedores:** La "Adapter Layer" contiene adaptadores específicos para cada core legacy y un cliente API para el core digital.
- **En el Diagrama de Componentes:** El "Core Router" dentro del Orchestrator Service decide el destino de cada transacción.
- **En el Event Backbone:** Eventos como account.created incluyen metadata indicando el core de origen, permitiendo que consumidores downstream actúen en consecuencia.

9. GESTIÓN DE IDENTIDAD Y ACCESO (IAM)

Flujo jerárquico de autenticación, autorización y control de acceso basado en políticas cero-trust para el ecosistema de canales e integraciones externas. Sus puntos más destacados son:

- **Autenticación Adaptativa por Perfil de Actor:** Se segregan los mecanismos de entrada de acuerdo con la naturaleza del usuario: autenticación ligera con OIDC/MFA para clientes retail o corporativos, protocolos corporativos SAML/OIDC para empleados internos, y estándares avanzados de seguridad financiera con OAuth 2.0 + FAPI junto con mTLS para proveedores externos (*Fintechs TPP*).
- **Centralización de Identidad (IdP):** El componente *Identity Provider* (Azure AD B2C / Keycloak) maneja la autenticación multifactor, la federación de identidades, la gestión de sesiones y el motor de autenticación adaptativa basada en riesgo (*risk-based adaptive authentication*), emitiendo tokens JWT firmados digitalmente.
- **Enforcement de Políticas con Open Policy Agent (OPA):** El flujo transaccional fluye desde el *API Gateway* (encargado de validar la integridad del JWT y extraer los *claims*) hacia el *Policy Decision Point* mediante OPA. Este componente evalúa de manera dinámica reglas en código (*Rego*), aplicando control de acceso basado en roles (RBAC) y atributos contextualmente financieros (ABAC) en cada microservicio mediante sidecars.



9.2 Mecanismos por Tipo de Actor

Actor	Autenticación	Autorización	Control de Acceso
Cliente Retail	OAuth 2.0 + OIDC + MFA (biometría móvil, OTP SMS)	Scopes: accounts:read, payments:write, profile:read	ABAC: límites por monto, horario, dispositivo
Cliente Empresa	OAuth 2.0 + OIDC + MFA + Certificado digital	Scopes: bulk-payments:write, payroll:execute	RBAC: roles corporativos (aprobador,

Actor	Autenticación	Autorización	Control de Acceso
			iniciador) + ABAC: montos
Empleado Interno	SAML 2.0 / OIDC + MFA (Azure AD)	RBAC: roles funcionales (cajero, analista riesgo, auditor)	Principio de mínimo privilegio, segregación de funciones
TPP (Open Finance)	OAuth 2.0 + FAPI + mTLS (certificado de cliente)	Scopes granulares por consentimiento del cliente	Rate limiting estricto, auditoría completa
Servicio-a-Servicio	mTLS (certificados X.509 gestionados por Istio)	Service Account con permisos mínimos	Network Policies + OPA policies

9.3 Provisioning y Lifecycle

- **SCIM 2.0** para provisioning automático de identidades desde Azure AD hacia los sistemas internos.
- **Just-In-Time (JIT) Provisioning** para accesos temporales.
- **Rotación automática de secretos** cada 24h vía HashiCorp Vault.
- **Revocación inmediata** de tokens ante detección de fraude (integración Fraud Detection → IdP).

10. ESTRATEGIA DE APIs INTERNAS Y EXTERNAS

10.1 Clasificación de APIs

Tipo	Audiencia	Estándar	Seguridad	Ejemplo
APIs Internas (Private)	Microservicios internos	OpenAPI 3.0 + gRPC (inter-service)	mTLS + Service Account	internal/accounts/v1/balance

Tipo	Audiencia	Estándar	Seguridad	Ejemplo
APIs de Partner (Partner)	Socios estratégicos (Banred, SWIFT)	OpenAPI 3.0 + ISO 20022	OAuth 2.0 + mTLS + IP Whitelist	partner/payments/v1/swift-transfer
APIs Abiertas (Open Finance)	TPPs autorizados	OpenAPI 3.0 + BIAN Service Domains	OAuth 2.0 + FAPI + Consent	open/v1/accounts/{id}/transactions
APIs de Canal (Experience)	Apps web y móvil propias	OpenAPI 3.0 (BFF pattern)	OAuth 2.0 + OIDC + Device binding	channel/v1/dashboard

10.2 Estándares de Mensajería

- Para APIs REST (Síncronas):

```
{
  "header": {
    :
    {
      "messageId": "uuid-v4",
      :
      "uuid-v4"
    },
    "correlationId": "uuid-v4",
    :
    "uuid-v4"
  },
  "timestamp": "2026-07-12T22:00:00Z",
  :
  "2026-07-12T22:00:00Z"
}
```

,

"source": "payment-execution-service",

:

"payment-execution-service"

,

"version": "1.0.0",

:

"1.0.0"

,

"channel": "mobile-app",

:

"mobile-app"

,

"sessionId": "session-uuid"

:

"session-uuid"

},

,

"data": {

:

{

"paymentId": "PAY-2026071200001",

:

"PAY-2026071200001"

,

"amount": { "value": 1500.00, "currency": "USD" },

:

{

"value"

:

1500.00

,

"currency"

:

```
"USD"  
}
```

```
,
```

```
  "debtor": { "accountId": "ACC-001", "name": "Juan Pérez" },  
  :  
  {  
    "accountId"  
    :  
    "ACC-001"  
    ,  
    "name"  
    :  
    "Juan Pérez"  
  }  
  ,
```

```
  "creditor": { "accountId": "ACC-002", "bankCode": "0010" },  
  :  
  {  
    "accountId"  
    :  
    "ACC-002"  
    ,  
    "bankCode"  
    :  
    "0010"  
  }  
  ,
```

```
  "purpose": "TRANSFER"  
  :  
  "TRANSFER"  
  },  
  ,
```

```
  "meta": {  
  :  
  {
```

```
  "pagination": { "page": 1, "size": 20, "totalElements": 150 }  
  :  
  {
```

```

"page"
:
1
,
"size"
:
20
,
"totalElements"
:
150
}
}
}

```

- **Para Eventos (Asíncronos) — CloudEvents Spec:**

```

{
  "specversion"
  :
  "1.0"
  ,
  "id"
  :
  "evt-uuid-v4"
  ,
  "source"
  :
  "urn:bank:payment-execution"
  ,
  "type"
  :
  "com.bank.payment.initiated.v1"
  ,
  "time"
  :
  "2026-07-12T22:00:00Z"
  ,
  "datacontenttype"
  :

```

"application/json"

,

"subject"

:

"PAY-2026071200001"

,

"data"

:

{

"paymentId"

:

"PAY-2026071200001"

,

"status"

:

"INITIATED"

,

"amount"

:

{

"value"

:

1500.00

,

"currency"

:

"USD"

}

,

"debtorAccountId"

:

"ACC-001"

,

"creditorAccountId"

:

"ACC-002"

```
}  
,  
  
  "extensions"  
  :  
  {  
  
    "correlationid"  
    :  
    "corr-uuid"  
    ,  
  
    "causationid"  
    :  
    "cause-uuid"  
    ,  
  
    "partitionkey"  
    :  
    "ACC-001"  
    ,  
  
    "schemaversion"  
    :  
    "1.0.0"  
  
  }  
}
```

10.3 Ejemplos de APIs por Dominio BIAN

Dominio BIAN	API	Método	Path	Descripción
Party Reference Data Directory	Consultar cliente	GET	/parties/{partyId}	Retorna datos maestros del cliente
Current Account	Consultar saldo	GET	/accounts/{accountId}/balance	Saldo disponible y contable
Payment Execution	Iniciar pago	POST	/payments	Inicia una transferencia
Payment Execution	Estado de pago	GET	/payments/{paymentId}/status	Consulta estado de la transacción
Consumer Loan	Simular crédito	POST	/loans/simulate	Simulación de cuotas y tasas
Fraud Resolution	Evaluar fraude	POST	/fraud/evaluate	Evaluación en tiempo real
Regulatory Compliance	Reporte regulatorio	POST	/compliance/reports	Genera reporte para regulador

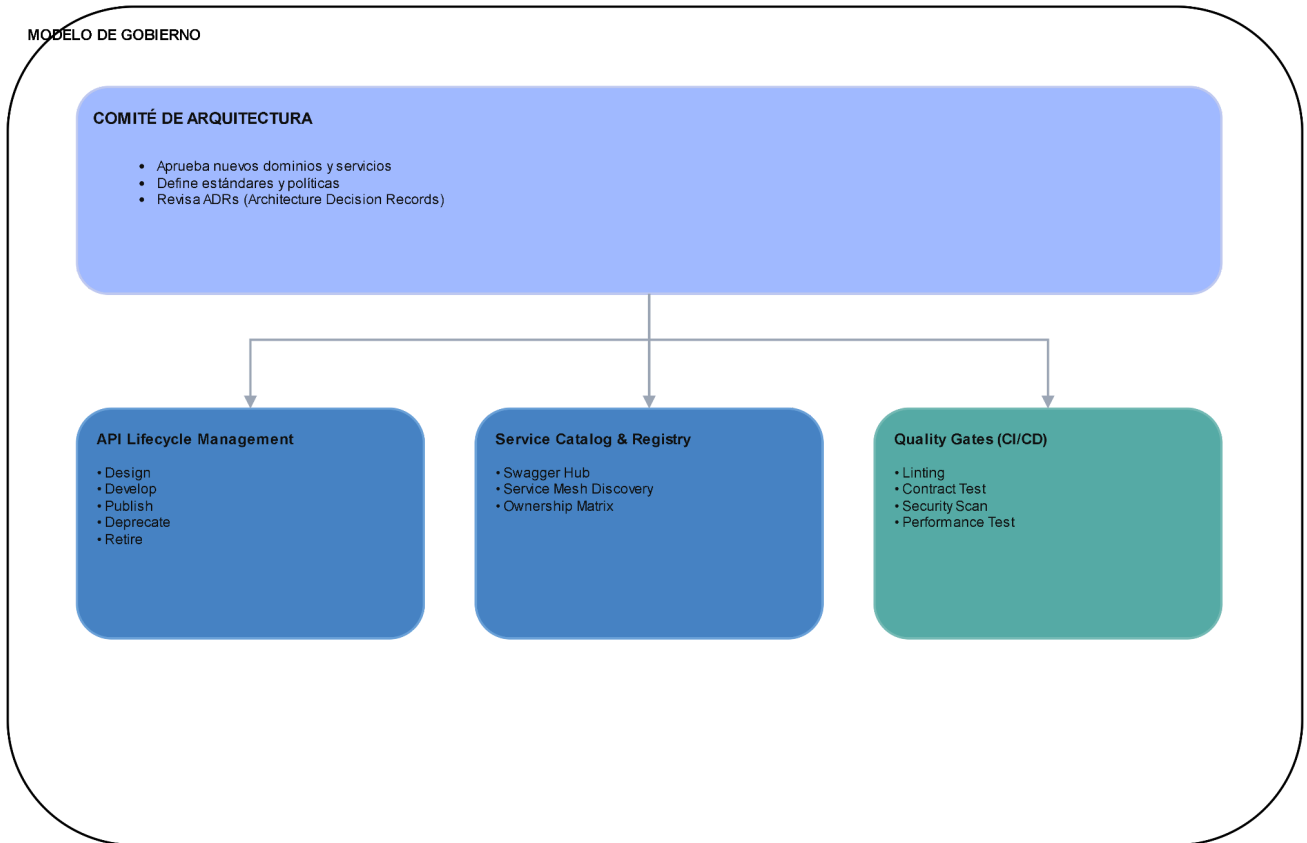
11. MODELO DE GOBIERNO DE APIs Y MICROSERVICIOS

11.1 Estructura de Gobierno

El diagrama expone la estructura jerárquica y operativa del gobierno de software bajo un comité centralizado, apoyado por mecanismos de control automatizado en el ciclo de vida, registro y despliegue. Sus aspectos principales son:

- **Comité de Arquitectura como Ente Rector:** Representa el nodo superior de toma de decisiones, encargado de aprobar nuevos dominios y servicios BIAN, definir las políticas tecnológicas transversales y auditar los Registros de Decisiones Arquitectónicas (ADRs).
- **Pilares Operativos del Gobierno:** Se despliega en tres bloques funcionales independientes:
 - *API Lifecycle Management:* Controla las etapas desde el diseño y desarrollo hasta la publicación, obsolescencia (*deprecate*) y retiro definitivo.
 - *Service Catalog & Registry:* Agrupa el catálogo activo en plataformas como Swagger Hub, el descubrimiento dinámico mediante Service Mesh y la matriz de propiedad de datos.
 - *Quality Gates (CI/CD):* Puertas de calidad automatizadas obligatorias que ejecutan linternas de código, pruebas de contrato de API, escaneos de seguridad y pruebas de rendimiento antes de autorizar el paso a producción.
- **API-First Design:** Publicación y aprobación previa en el *Service Catalog*.
- **Versionamiento Semántico:** Uso estricto de Major.Minor.Patch para compatibilidad.
- **Deprecation Policy:** Cabeceras Sunset y Deprecation activas por 6 meses antes del retiro.
- **Contract Testing (CDCT):** Uso de **Pact** para validación de contratos en CI/CD.

MODELO DE GOBIERNO



11.2 Políticas de Gobierno

Política	Descripción	Enforcement
API-First Design	Toda API se diseña (OpenAPI spec) antes de implementarse	PR review + linting en CI
Versionamiento Semántico	Major.Minor.Patch. Breaking changes solo en major version	Schema Registry validation
Deprecation Policy	Mínimo 6 meses de aviso antes de retirar una versión	API Gateway headers (Sunset, Deprecation)
Ownership Claro	Cada API/servicio tiene un equipo owner registrado en el catálogo	Service Catalog (Backstage/Compass)

Política	Descripción	Enforcement
Contract Testing	Consumer-Driven Contract Tests obligatorios antes de deploy	CI/CD gate (Pact)
SLA por Tier	Tier 1 (crítico): 99.99%, Tier 2: 99.95%, Tier 3: 99.9%	Monitoring + alertas